Web Application Penetration Test Report

Target: TryHackMe – “Neighbour” Room

Prepared by: Manoj Bora

Date: 23 August 2026

Classification: For Academic / Training Use Only

Document Control

Report Title	Web Application Penetration Test – TryHackMe Neighbour
Target	http://10.48.129.240 (TryHackMe lab instance)
Tester	Manoj Bora
Engagement Type	Black-box, authorized training lab (TryHackMe)
Date of Testing	23 August 2026
Date of Report	23 August 2026
Classification	For Academic / Training Use Only

Executive Summary

This report covers the TryHackMe “Neighbour” room, completed as part of the web module of my CEH practical coursework. I found three connected weaknesses on the site: sensitive information left inside the page’s source code, a working default login (guest:guest), and a broken access control issue that let me view the admin’s page just by changing a URL.

Using all three together, I was able to go from a normal guest login to full admin access and capture the room’s flag — all with just a browser, no special tools needed. The full technical details and risk ratings for each issue are covered in the Findings and Risk Summary sections below.

Scope & Objectives

Target: TryHackMe machine at 10.48.129.240.

Type of test: black-box — no credentials or information were given beforehand; everything was found during testing.

Objective: find and exploit web vulnerabilities to gain access, and escalate to admin if possible, as required for the CEH web module practical.

Methodology

The assessment followed the standard CEH five-phase approach, scoped down to what was actually needed for a single web target:

- Reconnaissance – identifying the target and understanding what services it exposes.
- Scanning & Enumeration – port/service scanning and manual exploration of the web application.
- Gaining Access – exploiting the identified weaknesses to obtain valid, then elevated, access.
- Maintaining Access / Covering Tracks – not applicable to this engagement; the objective was flag capture, not persistence.

1. Reconnaissance & Enumeration

I started the TryHackMe room and was assigned a target IP. A full TCP port scan was run with Nmap to identify what services were exposed before touching anything in a browser:

```
nmap -sV -p- 10.48.129.240
```

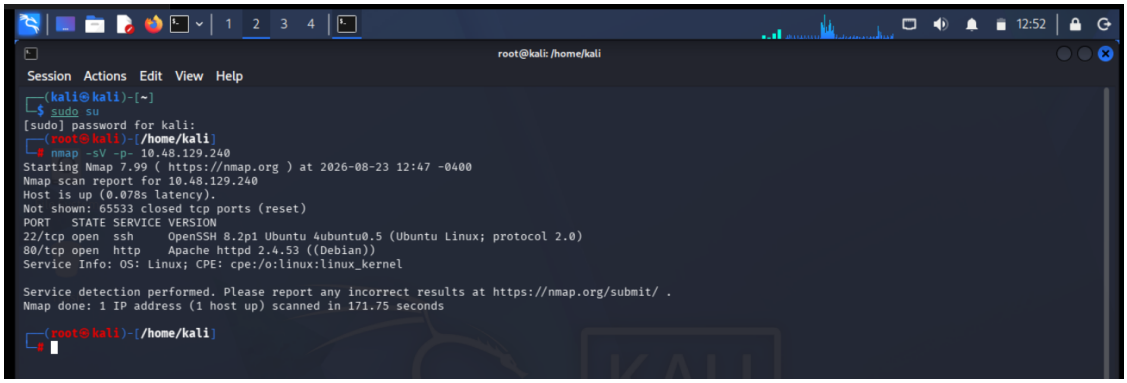


Figure 1 – Nmap scan showing SSH (22) and an Apache web server (80) open on the target

The scan showed two open ports: 22/tcp running OpenSSH 8.2p1 and 80/tcp running Apache httpd 2.4.53 on Ubuntu/Debian. Since this is a web-focused room, port 80 was the logical next step.

Tools Used

Tool	Type	Purpose
Nmap	Network scanner	Port scanning and service/version detection on the target host
Firefox (Kali)	Browser	Manual application walkthrough and viewing page source
Browser DevTools (Ctrl+U)	Built-in browser tool	Reviewing client-side HTML source for hidden comments/logic

Findings

I found three connected issues during this room. They're listed one by one below, but really they form one chain — the first one led to the second, and the second led to the third, which gave full admin access. Severity ratings and CVSS scores for all three are in the Risk Summary section at the end.

Finding 1: Sensitive Information Left in the Website's Source Code

When I opened the website and checked its source code (Ctrl+U), I found a comment left inside the code. This comment gave away a working username and password (guest / guest), and also mentioned that there's another account called "admin" that isn't supposed to be used yet.

What I did:

- Opened http://10.48.129.240/ in Firefox — a normal-looking login page appeared.
- Right-clicked and viewed the page source (Ctrl+U).

- Found a comment saying to use guest:guest to log in, and that the admin account is off-limits.

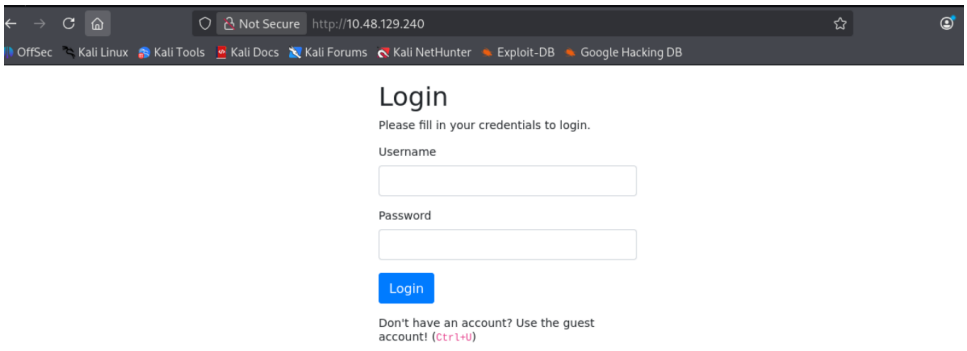


Figure 2 – The login page

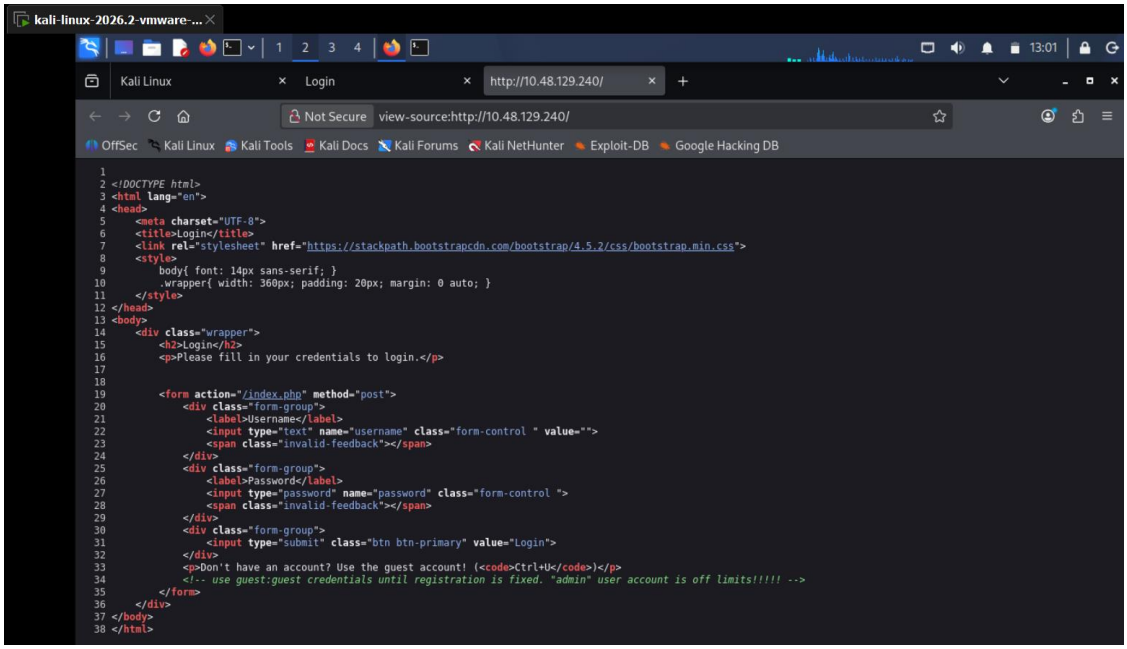


Figure 3 – Page source showing the guest:guest login and a mention of the admin account

Why it matters: Anything written in a website's code can be seen by anyone who checks it — including passwords and hints about other accounts. This one comment is what made the rest of the attack possible.

How to fix it: Don't leave passwords or internal notes inside the website's code. Anything sent to the browser should be treated as public, so this kind of information should only ever stay on the server.

Finding 2: Weak Login – Default Username and Password

The website had a default login (guest / guest) that was never changed or removed. Anyone who comes across these details — like I did from the source code — can log straight in without registering an account.

What I did:

- Typed “guest” as the username and “guest” as the password on the login page.
- Logged in right away, with no extra checks or verification of any kind.
- Landed on a profile page confirming I was logged in as guest.

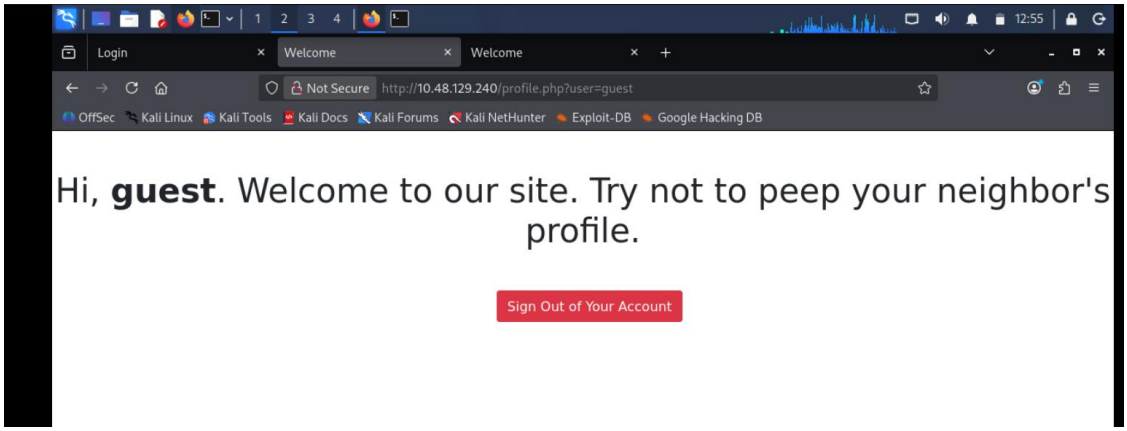


Figure 4 – Logged in successfully as guest, at profile.php?user=guest

Why it matters: A default login that’s never removed lets anyone in without any real effort. This became the way into the site, which then made the next issue possible.

How to fix it: Remove default logins like guest/guest before the site goes live. Every user should register properly, and passwords should never be left as simple defaults.

Finding 3: Changing the URL to Access the Admin Account

After logging in as guest, I noticed the page URL was profile.php?user=guest. The page even joked about not “peeping your neighbor’s profile” — which was a bit of a hint. Since I already knew from the source code that an admin account existed, I simply changed “user=guest” to “user=admin” in the address bar.

What I did:

- While logged in as guest, noted the URL: profile.php?user=guest.
- Changed it directly in the address bar to profile.php?user=admin.
- The page loaded the admin’s profile straight away, along with the room’s flag — no admin password needed.

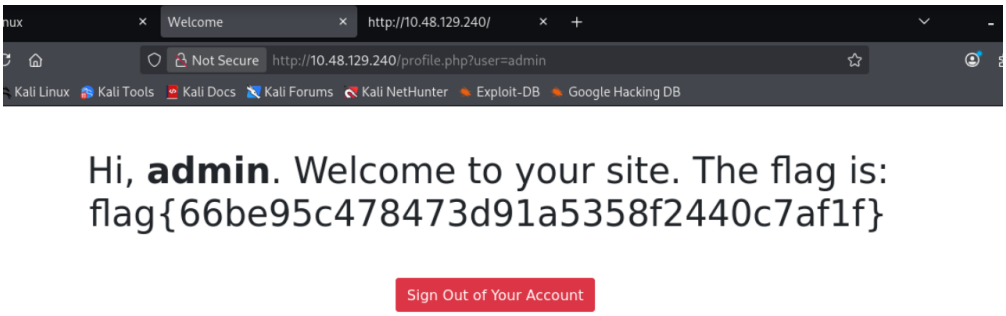


Figure 5 – Changing the URL to user=admin reveals the admin's page and the flag

Why it matters: This is the main issue in the room. The site decides whose profile to show based only on the URL, not on who is actually logged in. That means anyone can view someone else's account just by editing one word in the browser — which is how a simple guest login turned into full admin access.

How to fix it: The website should check who is really logged in on the server side, not just trust whatever username is typed in the URL. Even if the URL is changed, the server should refuse to show another user's data unless that person is actually allowed to see it.

Risk Summary

Finding	CVSS 3.1 Base	Assessed Severity	Category
Information Disclosure via Client-Side Source Code	5.3 (Medium)	Medium	CWE-Info Exposure
Broken Authentication – Weak / Hardcoded Credentials	7.5 (High)	High	Broken Auth
Broken Access Control – IDOR (Privilege Escalation)	6.5 (Medium)	High*	Broken Access Control

** Finding 3's CVSS base score reflects the isolated technical flaw; the assessed severity accounts for the fact that this is the step that delivers full administrative access.*

Conclusion & Recommendations

The "Neighbour" website can be fully taken over — from an anonymous visitor to full admin access — using nothing more than a browser and the "view page source" option. None of the three issues needed any special tools, which shows how a few small mistakes can add up to a serious problem.

What should be fixed first:

- The URL-based access issue (Finding 3) — this is what actually let a guest account see admin data.

- The default guest:guest login (Finding 2) — removing this closes the easy way in.
- The leftover comment in the source code (Finding 1) — a simple cleanup fix.

Overall, the site's security is weak. All three issues come down to the same root cause: trusting things it shouldn't — trusting a comment to stay hidden, trusting a default password to be temporary, and trusting a URL to be honest. Proper checks on the server side would have stopped this whole chain.

What I learned: this room showed that recon isn't just about scanning ports — checking the page source carefully was actually the most useful step here. It was also a good example of how small issues, like a leftover comment or an unchanged password, can combine into something much bigger when the server doesn't properly check who's allowed to see what.